

Translated

From: e-maxx.ru

Bellman-Ford Algorithm

Single source shortest path with negative weight edges

Suppose that we are given a weighted directed graph G with n vertices and m edges, and some specified vertex v . You want to find the length of shortest paths from vertex v to every other vertex.

Unlike the Dijkstra algorithm, this algorithm can also be applied to graphs containing negative weight edges. However, if the graph contains a negative cycle, then, clearly, the shortest path to some vertices may not exist (due to the fact that the weight of the shortest path must be equal to minus infinity); however, this algorithm can be modified to signal the presence of a cycle of negative weight, or even deduce this cycle.

The algorithm bears the name of two American scientists: Richard Bellman and Lester Ford. Ford actually invented this algorithm in 1956 during the study of another mathematical problem, which eventually reduced to a subproblem of finding the shortest paths in the graph, and Ford gave an outline of the algorithm to solve this problem. Bellman in 1958 published an article devoted specifically to the problem of finding the shortest path, and in this article he clearly formulated the algorithm in the form in which it is known to us now.

Description of the algorithm

Let us assume that the graph contains no negative weight cycle. The case of presence of a negative weight cycle will be discussed below in a separate section.

We will create an array of distances $d[0 \dots n-1]$, which after execution of the algorithm will contain the answer to the problem. In the beginning we fill it as follows: $d[v] = 0$, and all other elements $d[i]$ equal to infinity ∞ .

The algorithm consists of several phases. Each phase scans through all edges of the graph, and the algorithm tries to produce **relaxation** along each edge (a,b) having weight c . Relaxation along the edges is an attempt to improve the value $d[b]$ using value $d[a] + c$. In fact, it means that we are trying to improve the answer for this vertex using edge (a,b) and current answer for vertex a .